동국대학교
dongguk university

응용수치해석 및 실습 제14주

권기운

Chapter 10. 딥러닝 – 영상 분류

- 영상 처리에서 딥러닝
- CNNs
- Tensorflow 또는 Keras로 MNIST 영상 분류

인기 있는 심층 CNN

영상처리에서 딥러닝

- 기계학습의 주요 목표는 일반화
  - 학습데이터셋에 알고리즘을 학습
  - 그 알고리즘이 미지의 데이터 셋에 대해 높은 성능(정확도)를 갖기를 원함

- 영상 분류
  - 복잡한 영상처리
  - 학습데이터가 많을 때, 과적합 주의 : 정규화 사용
  - ML 모델의 일반화 능력이 좋아질 필요가 있음

- 전통적인 ML 기법을 사용하면 막대한 학습 데이터로 인해 매우 비싼 비용 듬
  - 특정 시점에서 일반화가 개선되지 않음
  - 관련 분야 많은 전문 지식과 인간의 개입이 요구됨
  - 단지 설계된 것만 수행
  - 딥러닝 개발의 필요성 요구됨

딥러닝이란 무엇인가?

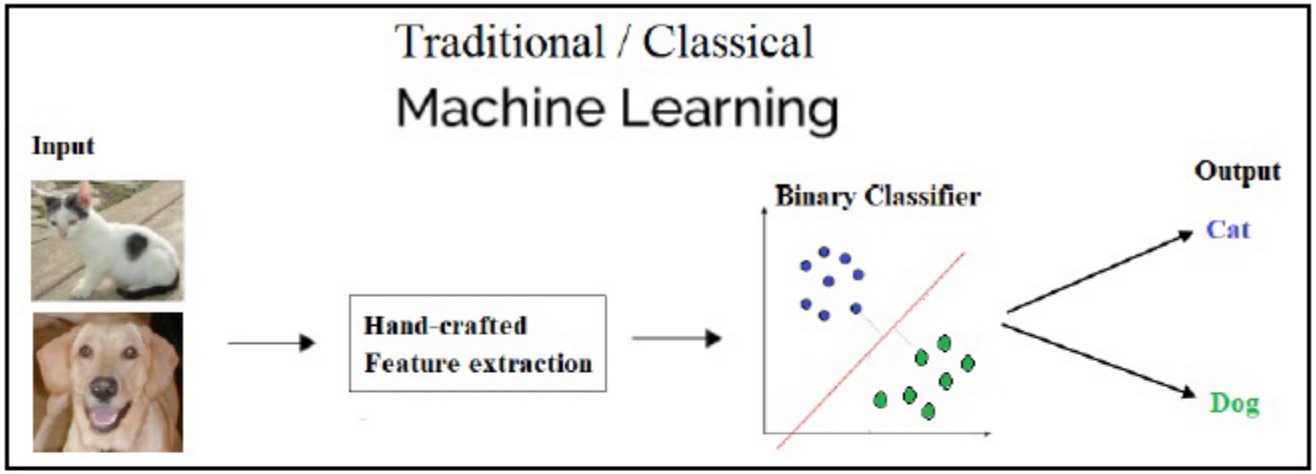
- 딥러닝
  - 기계학습(ML)의 부분집합
  - 인공신경망(ANN)과 비슷
    - 계단식 다중 계층(multi layer) 비선형 단위들을 사용
    - 뇌(뉴런)의 구조와 기능에 영감을 받은 알고리즘 사용
    - 연속된 계층(layer)은 이전 계층의 출력을 입력으로 삼음
  - 특징 추출, 변환, 데이터 처리, 패턴 찾기, 추상화 개발을 위해 사용
  - 지도(Supervised)와 비지도(Unsupervised) 학습에 사용
  - 경사하강(Gradient Descent) 알고리즘 사용하여 계층구조의 레벨과 다른 구조의 레벨들의 추상화에 상응하는 다층의 레벨로 표현하여 학습
  - 추상적인 표현을 덜 추상적인 표현으로 계산하고 단순화하여 중첩된 계층 구조로 세계를 표현하여 학습함으로써 막강한 힘과 융통성을 발휘
- 딥러닝과 영상분류
  - 은닉 계층 구조를 사용하여 점진적으로 영상 클래스들 학습
  - 점진적 특징 추출
    - 저수준 특징 추출 (밝거나 어두운 영역 식별)
    - 고수준 특징 (에지 등) 추출
    - 최고 수준 특징(모양) 추출
- 모든 노드 혹은 뉴런은 전체 영상의 작은 부분
  - 이들을 조합하면 전체 영상 묘사 : 영상을 완벽하게 표현
  - 네트워크의 모든 노드에 가중치 할당
  - 가중치는 출력과의 관계 강도
  - 모델이 개발되면서 가중치 조정됨

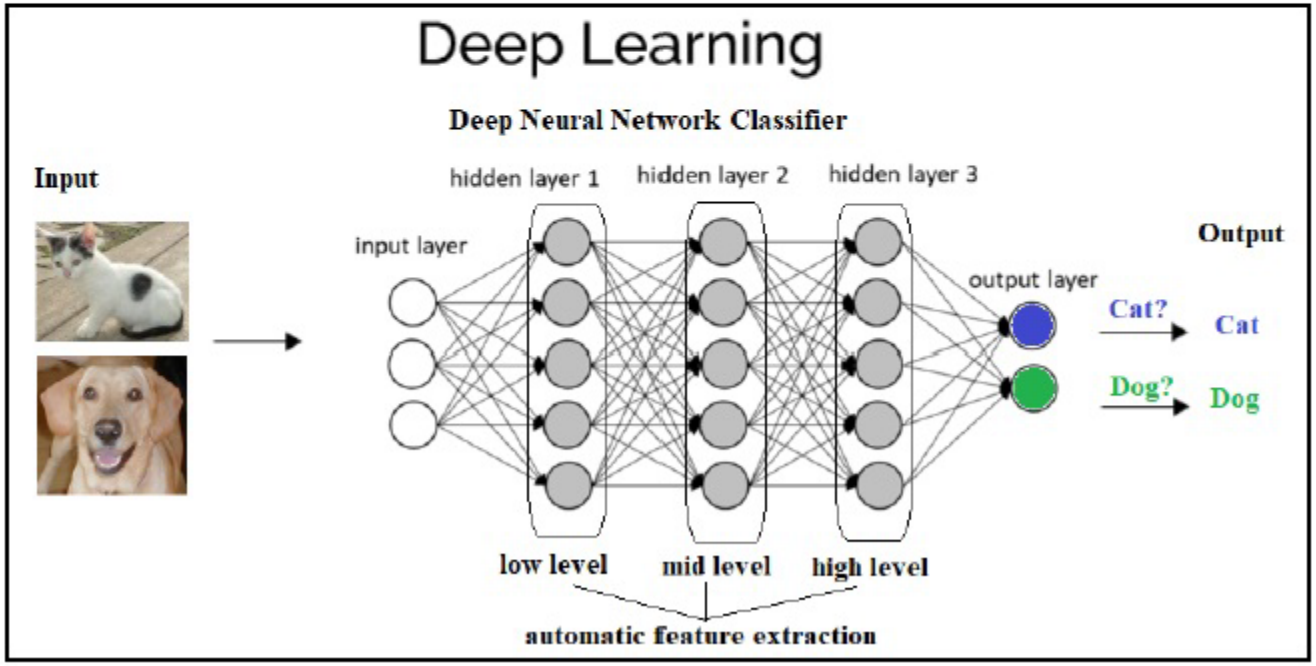
기존 학습 대 딥러닝

- 수작업 대 자동 특징 추출
  - ML : HOG, SIFT와 같은 전처리 수작업 : 영상의 복잡성 줄임, 학습 알고리즘 수행을 도움
  - DL : 점진적으로 낮은 수준과 높은 수준의 특징 학습
  - DL : 추출이나 작업(engineering)에 직접 설계한 특징(handcrafted feature)가 필요하지 않음
- 부분별 대 엔드투엔드 솔루션
  - ML : 단계문제로 해결 : 문제를 중간 단계들로 나누고 각 단계들을 해결한 다음 최종 결과를 종합하여 해결
  - DL : End-to-end 접근법
  - 객체 검출 문제
    - SVM : 경계박스 객체 탐지 알고리즘 : HOG를 가지는 모든 객체 우선 인식 : part by part
    - YOLO : 영상을 입력, 개체의 위치와 이름이 출력 : end-to-end
- 학습 시간과 고급 하드웨어
  - DL : 많은 수의 매개 변수와 거대한 데이터셋으로 인해 학습에 오랜 시간 걸림
    - GPU와 같은 고급 하드웨어 사용
    - 합리적 시간동안 학습
    - 시간은 모델을 효과적으로 학습하는 데 매우 중요한 요소
- 적응력과 전이가능
  - DL : 광범위한 응용과 다양한 영역에 적용
    - 전이학습 transfer learning
    - 동일한 분야 내에서 서로 다른 응용에 대해 사전 훈련된 심층망 사용
    - 사전 훈련된 영상 분류망은 객체망과 분할망을 검출하는 특징 추출 전처리자 frontend로 사용

고양이와 개 영상 분류

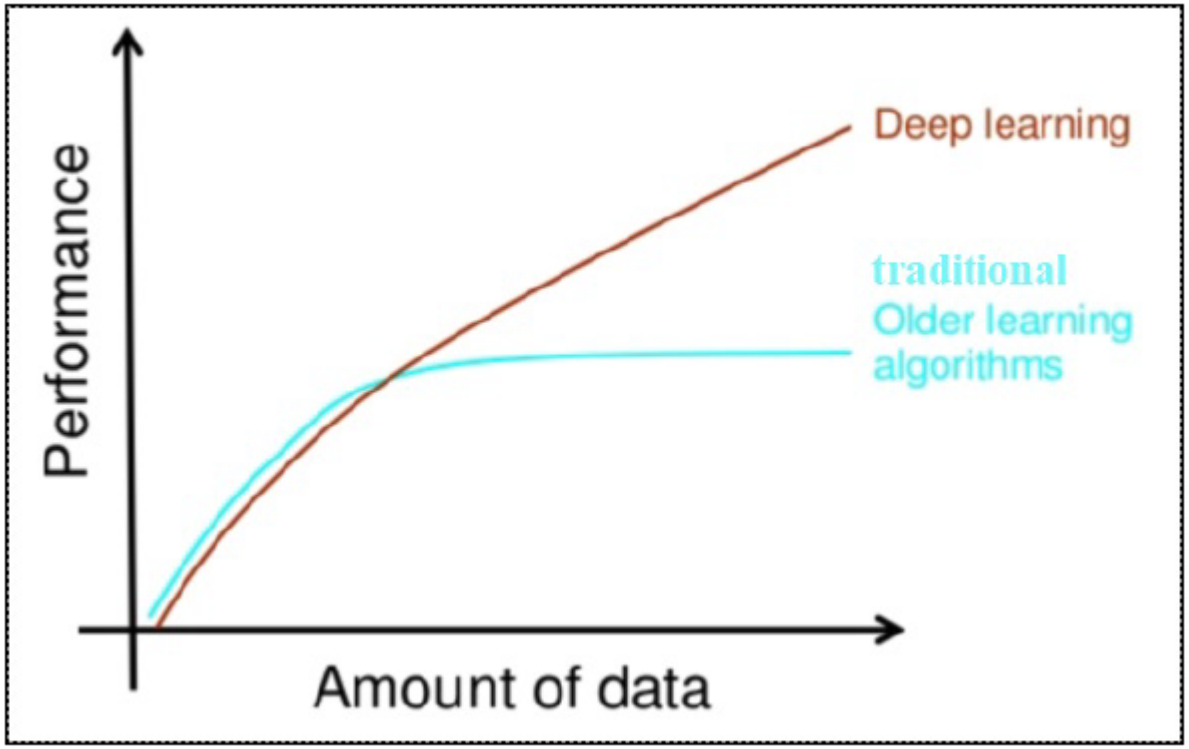
- ML : 특징 추출과 분류자
- DL : 은닉층과 의사결정





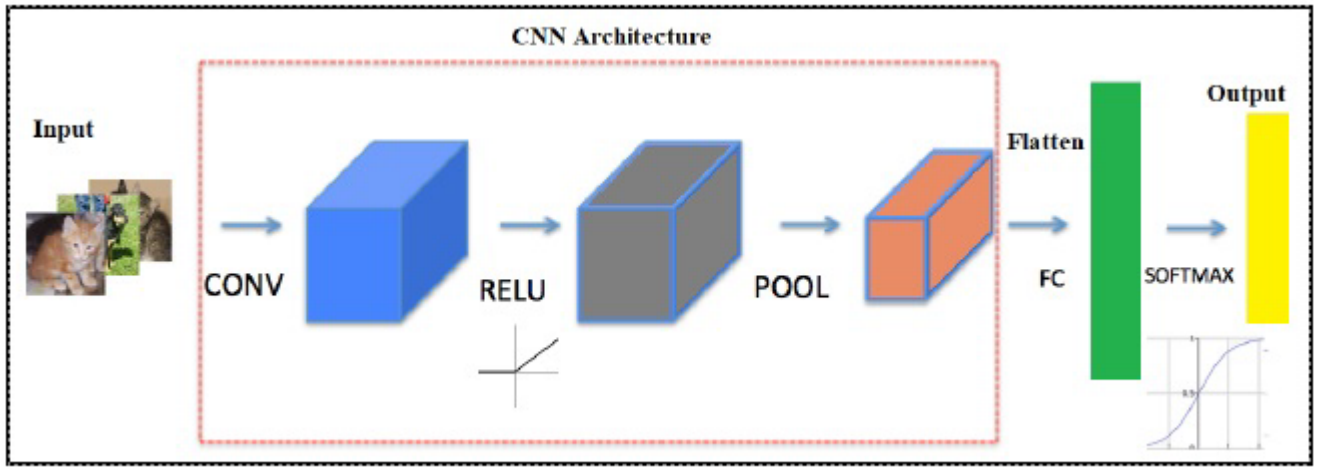
왜 딥러닝인가?

- 많은 데이터가 있으면 심층 신경망을 선택하는 것이 최선
- 풍부한 데이터가 더 나은 성능을 발휘
- 많은 시간과 더 많은 데이터를 사용할 수록 결과는 더 정확해 짐
- 고전적 ML : 복잡한 알고리즘 필요, 더 많은 데이터는 오히려 정확성을 저해
  - 정확도가 떨어지는 것을 보완하려면 더 복잡한 방법 적용
  - 더 많은 학습 데이터가 추가되어도 어느 시점부터 성능 향상이 없어짐



CNN : Convolutional Neural Network

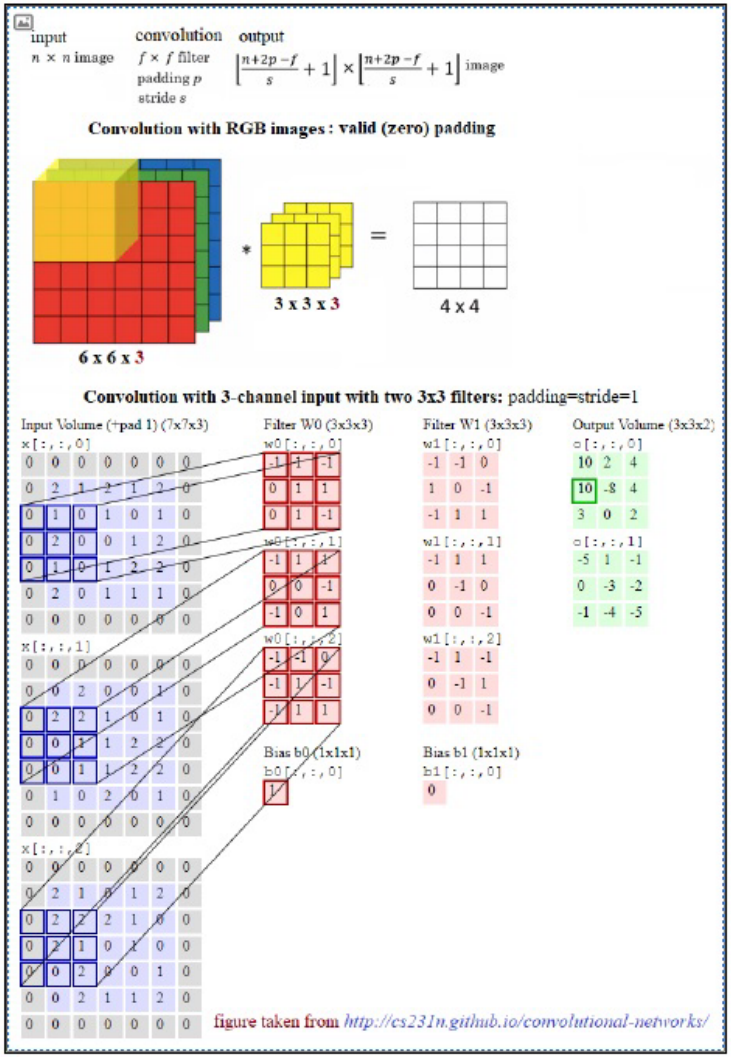
- 주로 영상을 입력으로 사용하는 심층 신경망
- 전통적 알고리즘에서 수 작업했던 hand-engineered했던 필터를 학습
- 특징 설계할 때, 사전 지식과 수 작업에서 독립
- 공유 가중치 아키텍쳐로 학습할 매개변수의 수를 줄이고 변환에 불변
- CNN의 일반적인 아키텍쳐와 작동 방식



- Convolution 계층, 비선형 ReLU 활성화 계층, Pooling 계층, 전체 연결 Fully Connected 계층, FC softmax 계층
- 영상분류를 위해 설계된 CNN
- 다중 컨벌루션, ReLU, 풀링 연속 계층 : 신경망을 더 깊게 만들어 복잡한 영상처리 가능

컨볼루션 계층

- CNN의 주요 구성 요소
- 여러 겹의 필터로 구성됨
- 컨볼루션필터를 입력영상에 적용, 특징맵 생성
- 입력 영상 위에서 필터를 슬라이딩하여 수행
- 너비, 높이, 깊이로 표현됨
  - 너비*높이 : 필터의 크기
  - 깊이 : 필터의 개수
- 보폭 stride : 각 단계에서 컨볼루션 필터의 이동 간격(기본값 1)
- 패딩 padding : 입력영상의 경계 주위를 둘러싸는 0의 계층
  - 입력영상과 출력영상의 크기를 같게 하려고 사용
  - 동일same 패딩
- 그림 : 3*3*3 컨볼루션 필터
  - 첫번째 : 유효valid 패딩(padding=0)
  - 두번째 : stride=2, padding=1, 두 개의 필터 W0,W1



풀링 계층, 비선형 ReLU 계층, 전체연결 계층

- 풀링 계층
- 컨볼루션 작업 후에 풀링 작업을 수행해서 매개 변수의 차원과 개수를 줄임
- 학습 시간을 줄임, 학습을 위한 필요 데이터를 줄임, 과적합에도 대처
- 각 특징 계층을 독립적으로 다운샘플링하여 높이와 너비는 줄이는 깊이는 그대로 유지
- 가장 일반적인 것은 최대값 풀링 : 윈도우 내의 최댓값을 취함
  - 입력 영상위로 윈도우를 슬라이딩하면서 윈도우 내의 최댓값 구함
  - 컨볼루션과 마찬가지로 윈도우 크기와 보폭 지정 가능

- 비선형 ReLU 계층
- 어떤 종류의 신경망이 강력해지려면 비선형성을 포함하여야 함
- 컨볼루션 연산의 결과는 비선형 활성화 함수를 통해 전달
- 가장 일반적인 것은 ReLU 함수
- 기울기 소멸 문제를 해결하기 위해서는 시그모이드 Sigmoid 활성화 함수를 사용
- 최종 특징값은 실제 합계에 ReLU 함수가 적용된 값

- 전체 연결 계층: FC : Fully Connected
- 컨볼루션과 풀링 계층 이후에 일반적으로 2개의 FC 계층
- 컨볼루션과 풀링 계층의 출력은 3차원이지만 FC 계층의 입력은 1차원
  - 최종 풀링 계층의 출력을 벡터로 전개 Flattening 하여 FC 계층의 입력으로 사용
  - Flattening : 단순히 3차원 볼륨의 숫자들을 1차원 벡터로 나열

드롭아웃, Dropout

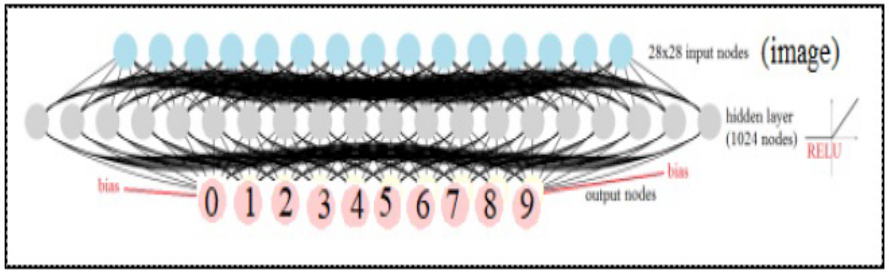
- 심층 신경망에 가장 널리 사용되는 정규화 기법
- 과적합 방지
- 미지의 데이터셋에서 딥러닝 작업의 성능을 높이기 위해 사용
- 학습 시간 동안 각 반복에서 한 뉴런을
  - 일시적으로 탈락시키거나
  - 확률p로 일부를 비활성화
- 현재 반복에서 이 뉴런에 대한 모든 입력과 출력을 비활성화
- Hyperparameter p : 드롭 아웃 비율 dropout rate
  - 일반적으로 0.5 : 뉴런들 중 50%가 탈락

Tensorflow 또는 Keras(Tensorflow.keras)로 영상 분류

- MNIST 필기 숫자 분류 문제를 해결
- 두 가지 유명한 딥러닝 라이브러리
- Tensorflow TF : 가장 유명
  - 매우 크고 멋진 커뮤니티
  - 사용하기가 쉽지 않다
  - 저수준 라이브러리 제공이 Keras 보다 더 유연
- Keras : TF 기반 고급 API
  - 저수준 아키텍쳐 제어 미흡
  - TF와 비교하여 사용하기 쉬움
  - 사용자 친화적

Tensorflow를 이용한 분류

- 아주 간단한 심층 신경망
  - FC – ReLU – softmax FC
- 입력 : 28*28 영상을 Flattened 하여 구성
- 은닉층 1024개 노드
- 출력 10개 노드



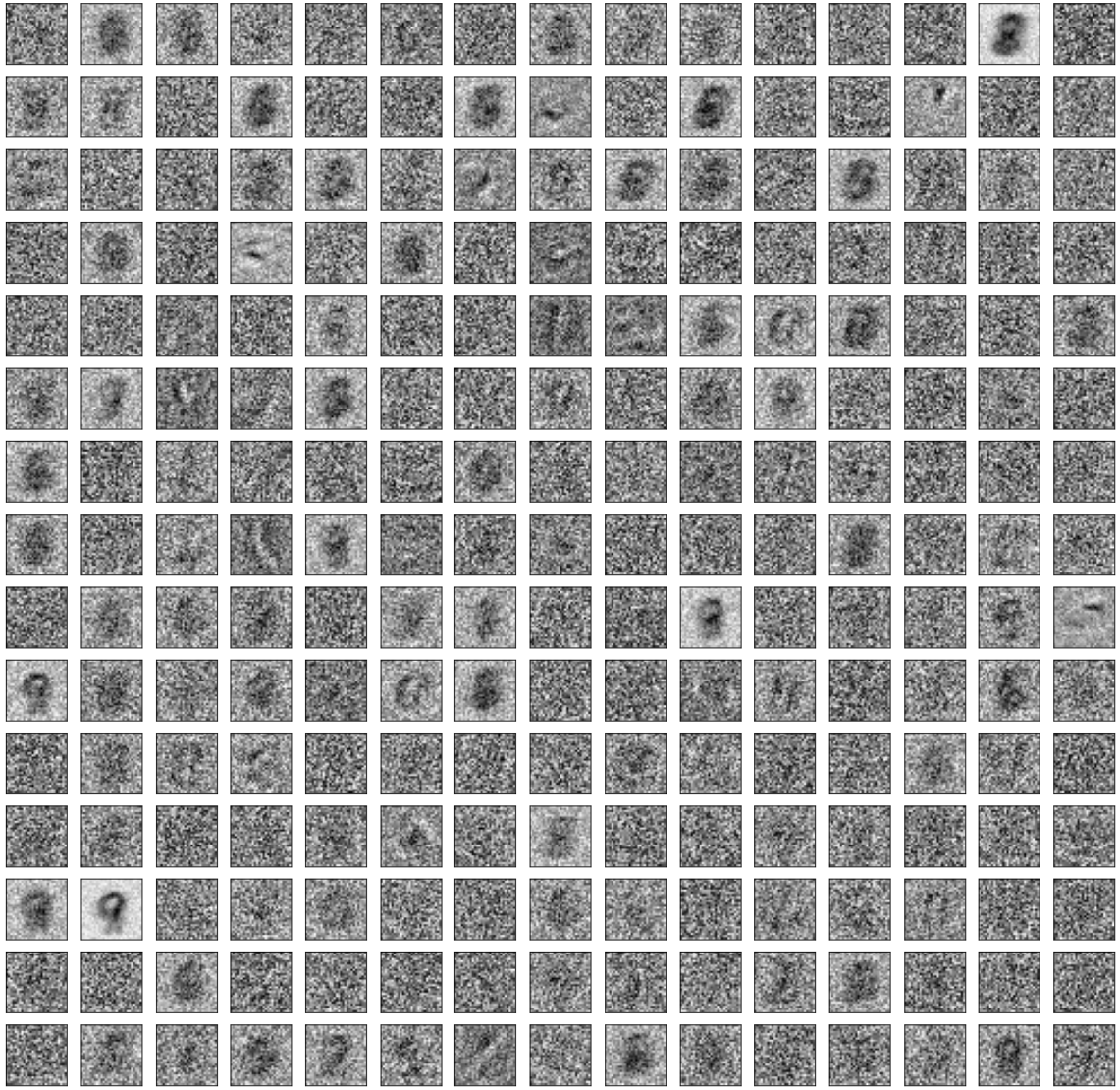
- MNIST 입력
- 학습 영상 두 부분으로 나눔 : 학습 5만개, 검증 1만개
- 레이블 : one-hot encoding을 이용하여 이진영상으로 구성
- Tensorflow 그래프, 변수, 상수, placeholder 텐서 초기화
- 두 개의 가중치 계층($\lambda_1 = \lambda_2 = 1$)에 L2 정규화 적용
- Softmax_cross_entropy_with_logits_v2( )함수 최소화 위해 배치크기 256의 학습 알고리즘 사용 : SGD(Stochastic Gradient Descent) 최적화
- TF 세션 객체를 6000 단계 미니배치로 실행
- 전파/역전파로 학습된 모델 업데이트
- 모델 평가 : 정확도 97%

Example 1. TF를 이용한 MNIST분류 : 96.3%

- Initialized
- Minibatch loss at step 0: 105038.359375
- Minibatch accuracy: 12.5%
- Validation accuracy: 10.1%
- Minibatch loss at step 500: 103796.421875
- Minibatch accuracy: 27.3%
- Validation accuracy: 29.4%
- Minibatch loss at step 1000: 46623.281250
- Minibatch accuracy: 30.5%
- Validation accuracy: 30.1%
- Minibatch loss at step 1500: 20943.210938
- Minibatch accuracy: 34.4%
- Validation accuracy: 35.6%
- Minibatch loss at step 2000: 9408.232422
- Minibatch accuracy: 45.7%
- Validation accuracy: 45.5%
- Minibatch loss at step 2500: 4226.496094
- Minibatch accuracy: 62.1%
- Validation accuracy: 66.3%
- Minibatch loss at step 3000: 1898.765625
- Minibatch accuracy: 77.3%
- Validation accuracy: 81.4%
- Minibatch loss at step 3500: 852.947876
- Minibatch accuracy: 87.5%
- Validation accuracy: 88.5%
- Minibatch loss at step 4000: 383.180817
- Minibatch accuracy: 92.6%
- Validation accuracy: 92.4%
- Minibatch loss at step 4500: 172.222565
- Minibatch accuracy: 96.1%
- Validation accuracy: 91.3%
- Minibatch loss at step 5000: 77.535873
- Minibatch accuracy: 94.5%
- Validation accuraccy: 95.3%
- Minibatch loss at step 5500: 34.833199
- Minibatch accuraaccuracyccy: 97.3%
- Validation accuracy: 96.0%
- Minibatch loss at step 6000: 15.769064
- Minibatch accuracy: 96.5%
- Validation accuracy: 96.2%
- Test accuracy: 96.3%

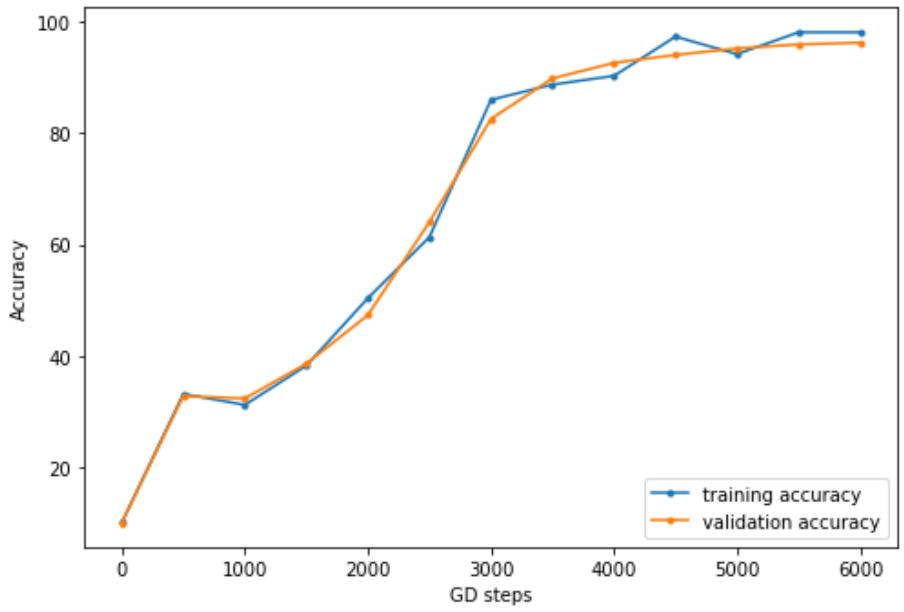
Example 1: Weights1

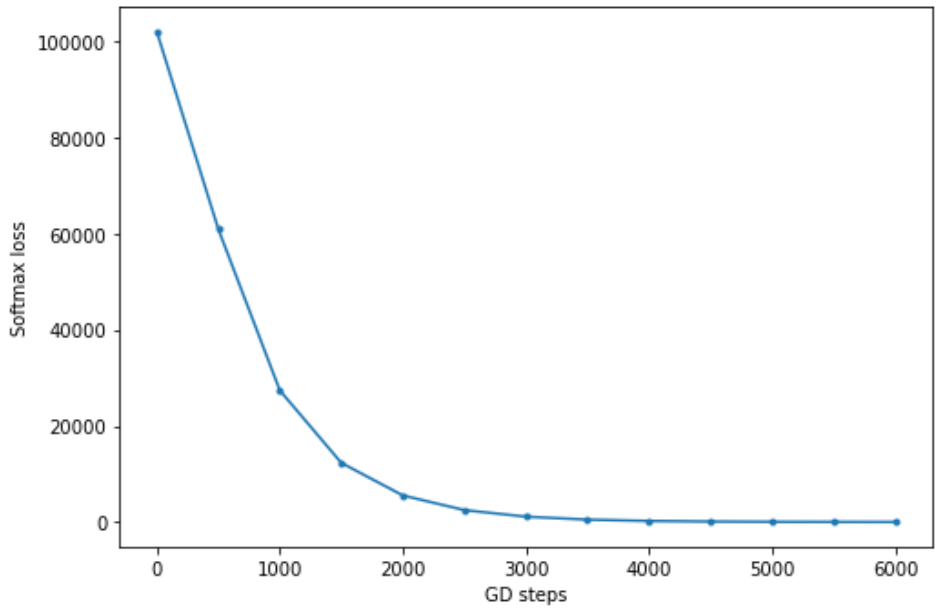
SGD after Setp 6000 with lambda1=lambda2=0.1



동국대학교
dongguk university

Example 1 : Training/Validation Accuracy, Softmax loss





동국대학교
dongguk university

Keras를 사용하여 밀집 FC 계층으로 분류

- 밀집 FC 계층만 사용하여 MNIST 분류 구현
- 은닉층을 하나 더, 드롭아웃 계층도 함께 사용
- Tensoflow.keras.models.Sequential( ) 함수 사용
- 순차적으로 계층을 추가
- 은닉층 두 개, 200개 노드와 드롭아웃 비율 15%
- Adam : SGD에 가속도 momentum 옵션 추가
- 10 epochs
- 간단한 아키텍쳐 변경으로 96.3% => 97% 정확도 향상

- Keras로 디자인한 신경망 구조 시각화

- 중간계층에서 학습한 가중치 시각화
  - 첫 밀집계층 : 200개 은닉 노드에 대한 가중치 시각화
  - 마지막 밀집계층 : 10개 은닉 노드에 대한 가중치 시각화

Example 2. Keras를 이용한 MNIST분류 : Model

Model: "sequential_2"

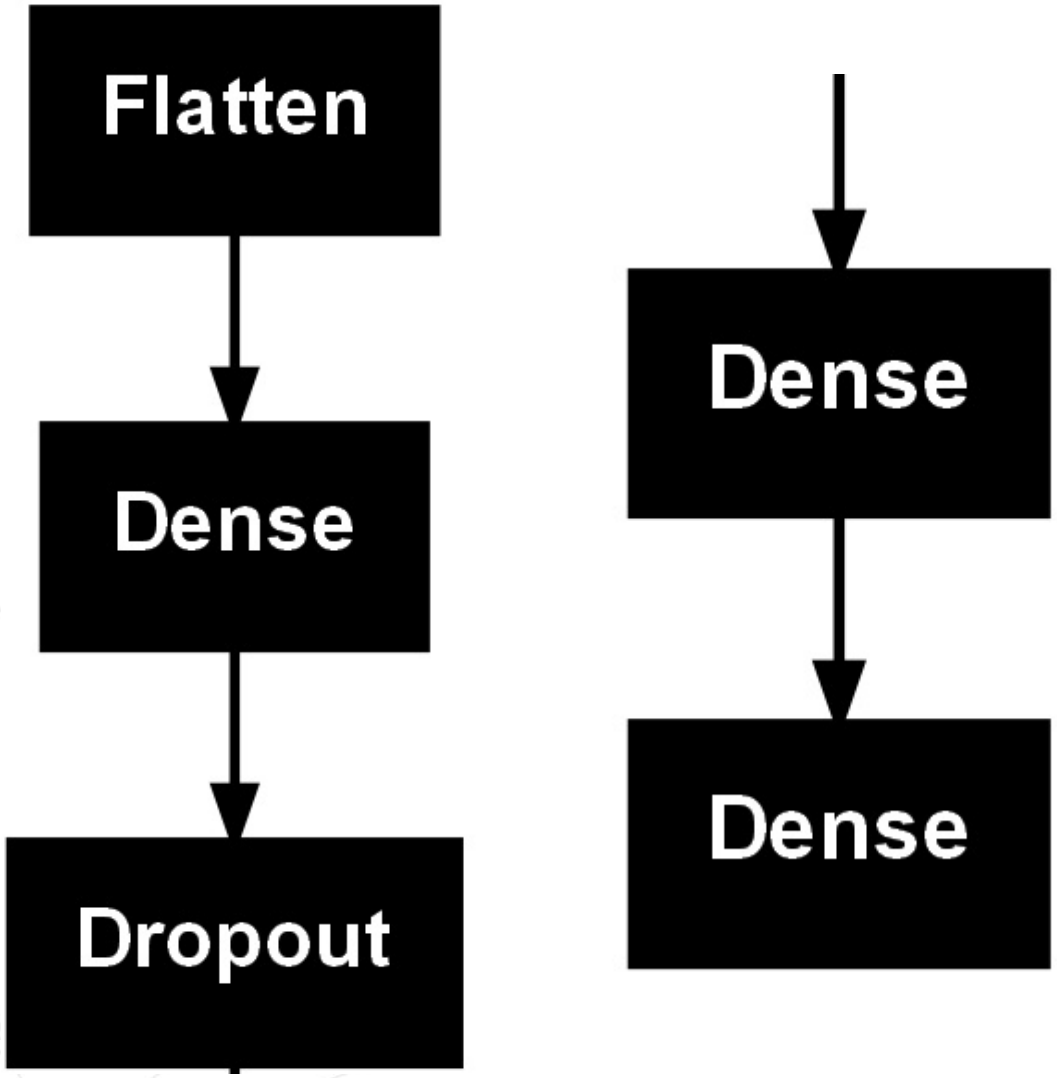
Layer (type)	Output Shape	Param #
flatten_2 (Flatten)	(None, 784)	0
dense_6 (Dense)	(None, 200)	157,000
dropout_2 (Dropout)	(None, 200)	0
dense_7 (Dense)	(None, 200)	40,200
dense_8 (Dense)	(None, 10)	2,010

Total params: 199,210 (778.16 KB)
Trainable params: 199,210 (778.16 KB)
Non-trainable params: 0 (0.00 B)
None

Example 2 : Fitting and Test Error

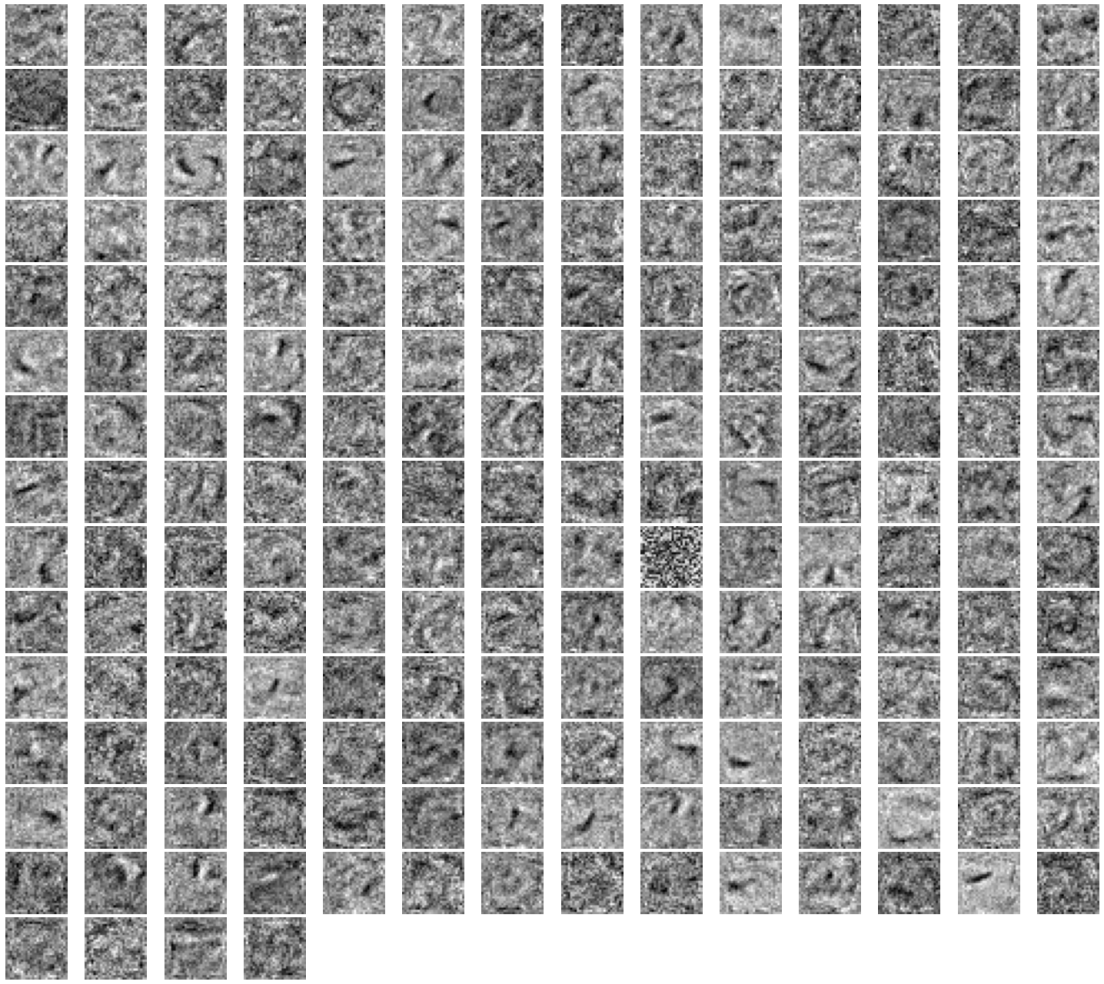
```
Epoch 1/10
300/300 - 1s - 5ms/step - accuracy: 0.9081 - loss: 0.3228 - val_accuracy: 0.9546 - val_loss: 0.1497
Epoch 2/10
300/300 - 1s - 2ms/step - accuracy: 0.9638 - loss: 0.1216 - val_accuracy: 0.9691 - val_loss: 0.1025
Epoch 3/10
300/300 - 1s - 2ms/step - accuracy: 0.9754 - loss: 0.0826 - val_accuracy: 0.9734 - val_loss: 0.0844
Epoch 4/10
300/300 - 1s - 2ms/step - accuracy: 0.9814 - loss: 0.0601 - val_accuracy: 0.9784 - val_loss: 0.0683
Epoch 5/10
300/300 - 1s - 2ms/step - accuracy: 0.9860 - loss: 0.0467 - val_accuracy: 0.9768 - val_loss: 0.0742
Epoch 6/10
300/300 - 1s - 2ms/step - accuracy: 0.9896 - loss: 0.0349 - val_accuracy: 0.9785 - val_loss: 0.0705
Epoch 7/10
300/300 - 1s - 2ms/step - accuracy: 0.9922 - loss: 0.0259 - val_accuracy: 0.9804 - val_loss: 0.0645
Epoch 8/10
300/300 - 1s - 2ms/step - accuracy: 0.9946 - loss: 0.0196 - val_accuracy: 0.9783 - val_loss: 0.0715
Epoch 9/10
300/300 - 1s - 2ms/step - accuracy: 0.9951 - loss: 0.0163 - val_accuracy: 0.9784 - val_loss: 0.0762
Epoch 10/10
300/300 - 1s - 2ms/step - accuracy: 0.9959 - loss: 0.0133 - val_accuracy: 0.9706 - val_loss: 0.1117

Accuracy: 0.9706000089645386
Error: 2.9399991035461426 %
```

Example 2 : Keras Plot



Example 2 : Weights Dense_6 (784*1=>28*28)

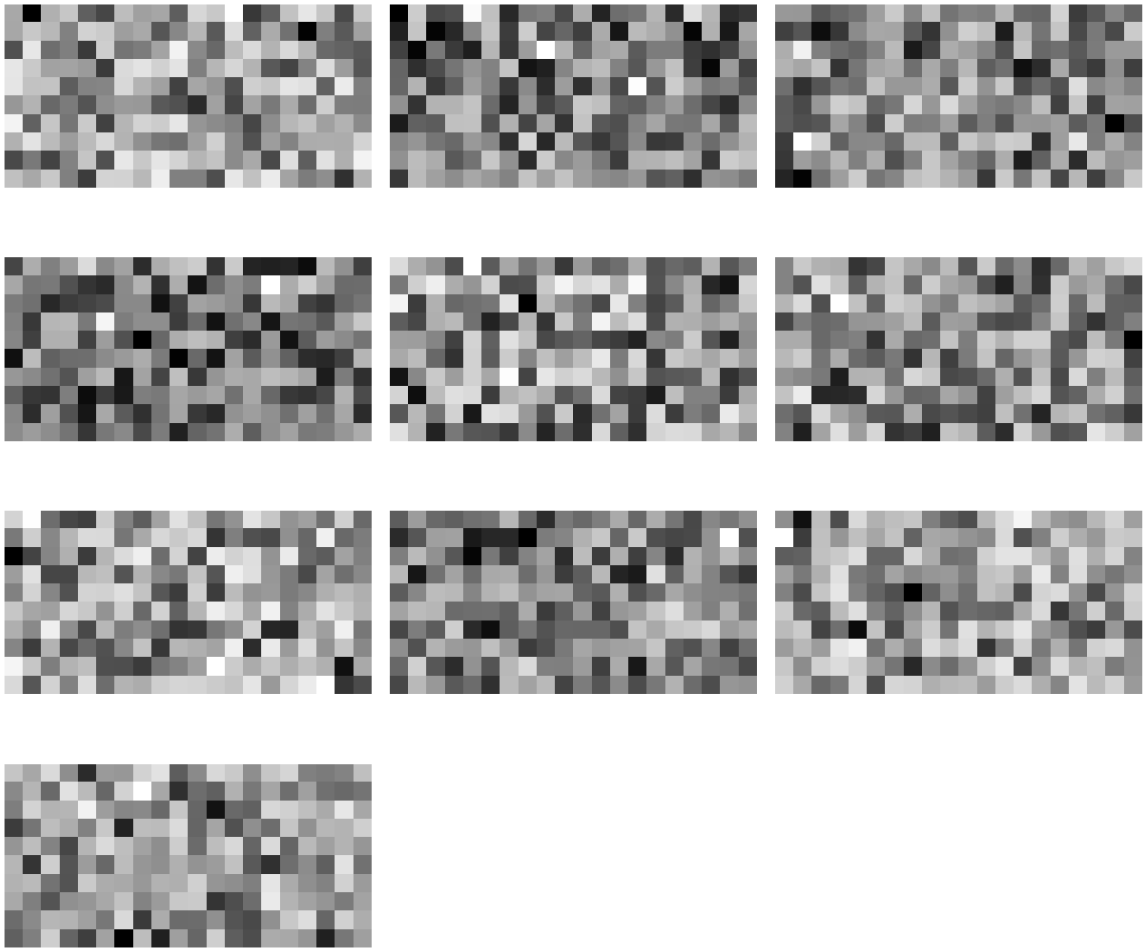
Dense_6 Weights (200 hidden units)



동국대학교
dongguk university

Example 2 : Weights Dense_8 (200*1)

Dense_8 Weights (10 hidden units)

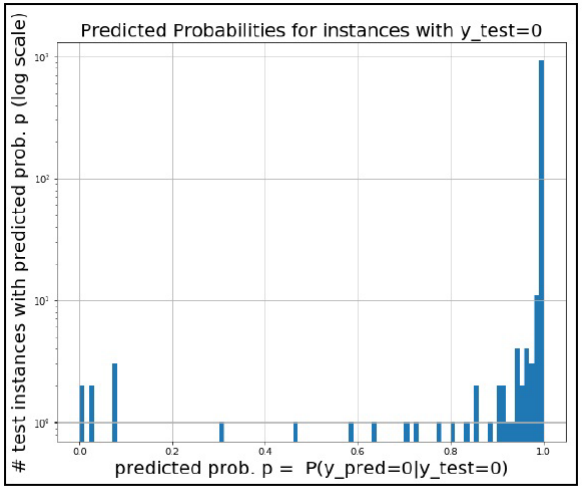


동국대학교
dongguk university

Keras로 분류를 위한 CNN

- 컨볼루션 계층과 풀링 계층이 있어야 함
- 이후 데이터는 1차원으로 전개
- 테스트 데이터 셋의 정확도 97% => 98.9%
- ReLU 활성화, 64개 필터의 5*5 컨벌루션 계층 도입
- 보폭 2인 2*2 최댓값 풀링 계층 사용 후 데이터를 1차원으로 전개
- 100노드로 구성된 은닉층 1개의 밀집 계층
- 출력 계층 : softmax 밀집 계층

- 레이블 0인 테스트 샘플에 대해 출력 클래스를 0으로 예측한 확률 분포 ➔



- 학습한 영상 특징 시각화
  - 64개 필터를 갖는 6개 특징 IND=0, IND=1

Example 3. Keras 분류를 위한 CNN : Model

Model: "sequential_3"

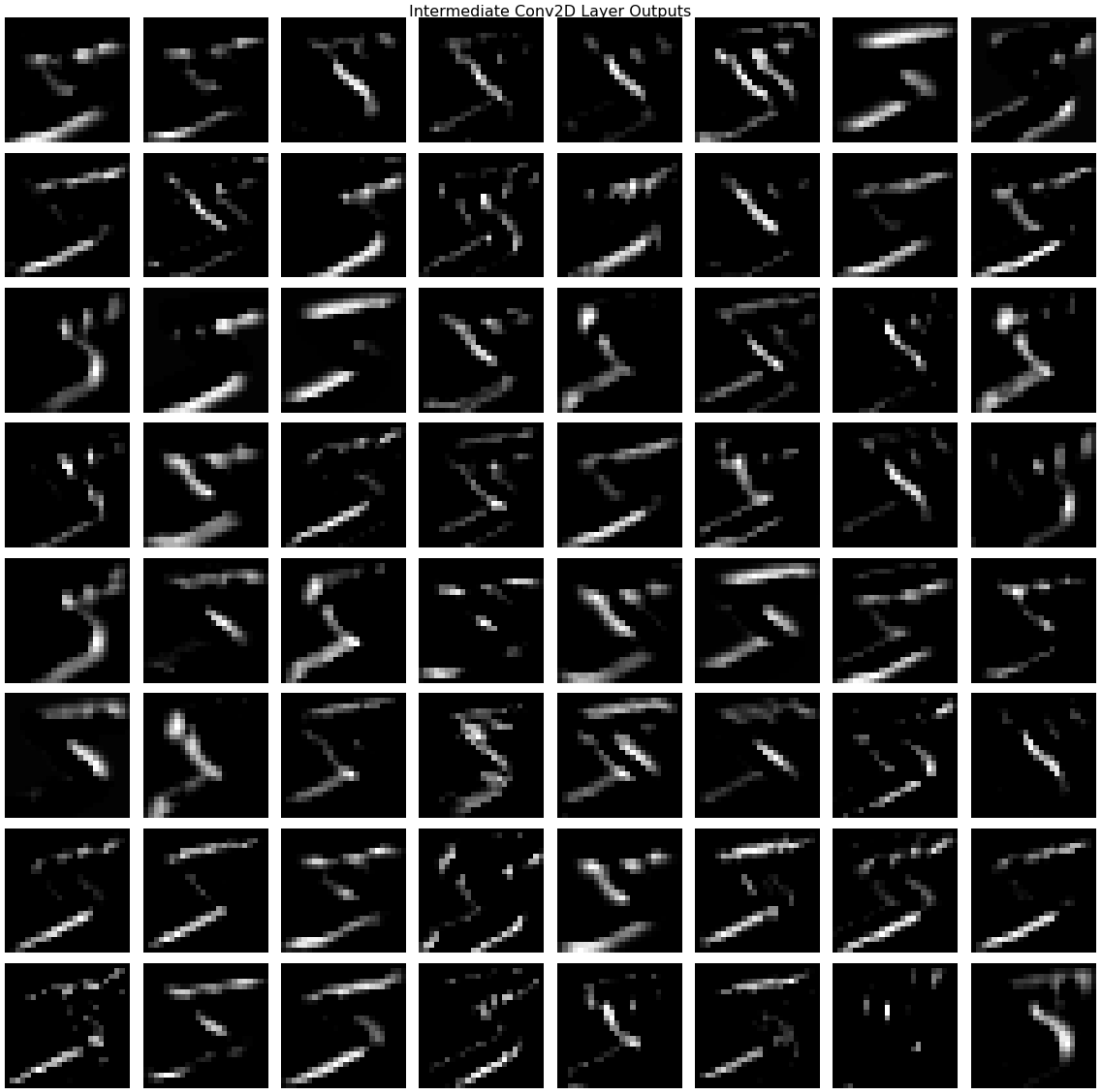
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 24, 24, 64)	1,664
max_pooling2d (MaxPooling2D)	(None, 12, 12, 64)	0
flatten_3 (Flatten)	(None, 9216)	0
dense_9 (Dense)	(None, 100)	921,700
dense_10 (Dense)	(None, 10)	1,010

Total params: 924,374 (3.53 MB)
Trainable params: 924,374 (3.53 MB)
Non-trainable params: 0 (0.00 B)

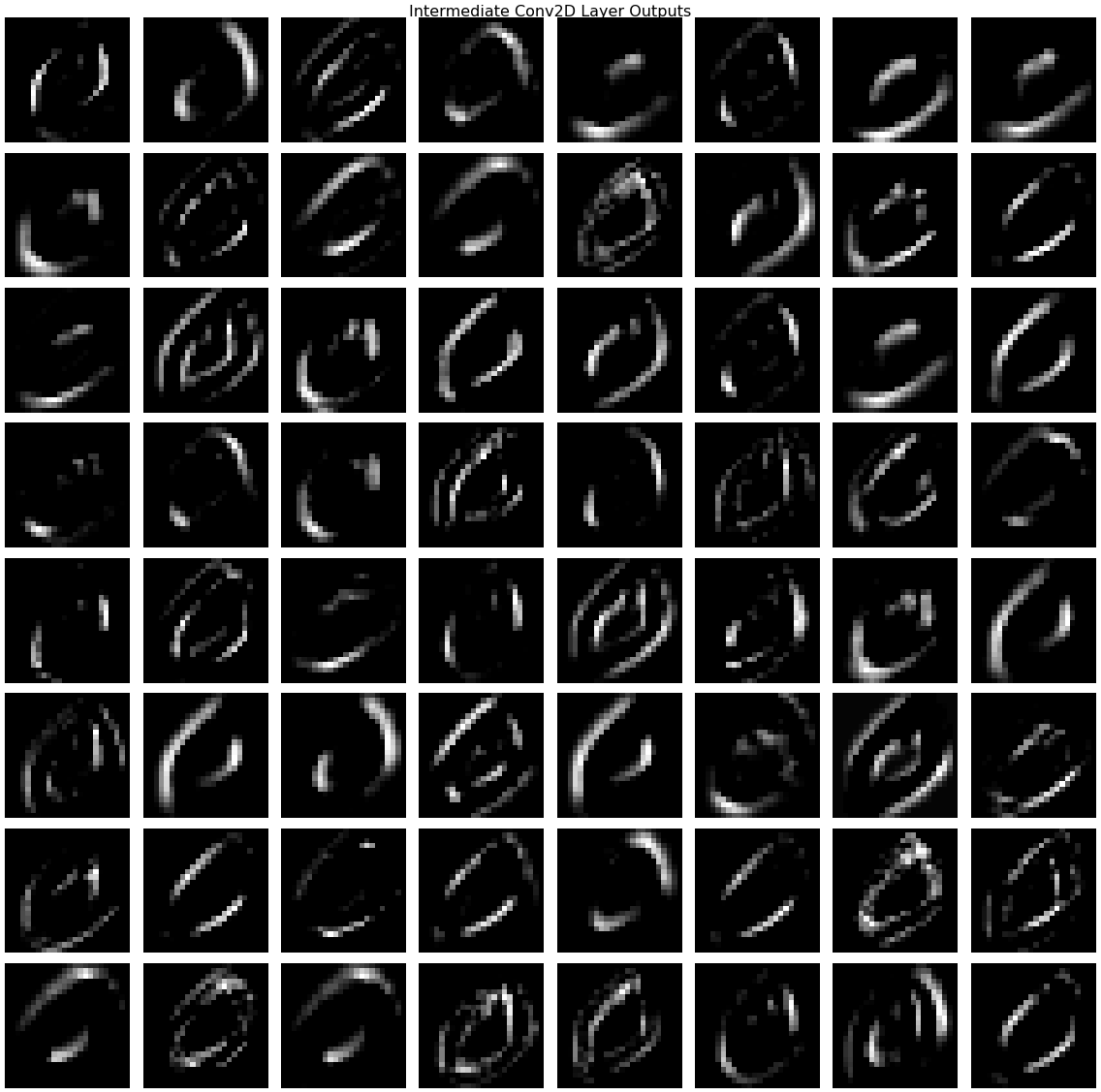
Example 3 : Fitting and Accuracy

```
Epoch 1/10
300/300 - 5s - 18ms/step - accuracy: 0.9411 - loss: 0.2072 - val_accuracy: 0.9768 - val_loss: 0.0741
Epoch 2/10
300/300 - 5s - 16ms/step - accuracy: 0.9816 - loss: 0.0596 - val_accuracy: 0.9843 - val_loss: 0.0499
Epoch 3/10
300/300 - 5s - 16ms/step - accuracy: 0.9872 - loss: 0.0407 - val_accuracy: 0.9866 - val_loss: 0.0386
Epoch 4/10
300/300 - 5s - 17ms/step - accuracy: 0.9903 - loss: 0.0310 - val_accuracy: 0.9873 - val_loss: 0.0364
Epoch 5/10
300/300 - 5s - 16ms/step - accuracy: 0.9929 - loss: 0.0234 - val_accuracy: 0.9888 - val_loss: 0.0323
Epoch 6/10
300/300 - 5s - 16ms/step - accuracy: 0.9947 - loss: 0.0170 - val_accuracy: 0.9885 - val_loss: 0.0357
Epoch 7/10
300/300 - 5s - 16ms/step - accuracy: 0.9958 - loss: 0.0132 - val_accuracy: 0.9888 - val_loss: 0.0356
Epoch 8/10
300/300 - 5s - 16ms/step - accuracy: 0.9961 - loss: 0.0119 - val_accuracy: 0.9893 - val_loss: 0.0344
Epoch 9/10
300/300 - 5s - 16ms/step - accuracy: 0.9973 - loss: 0.0094 - val_accuracy: 0.9889 - val_loss: 0.0378
Epoch 10/10
300/300 - 5s - 17ms/step - accuracy: 0.9981 - loss: 0.0065 - val_accuracy: 0.9880 - val_loss: 0.0379

Accuracy: 0.9890000224113464
Error: 1.0999977588653564 %
```

Example 3 : 중간 계층 시각화 : IND=0



Example 3 : 중간계층 시각화 : IND=1



동국대학교
dongguk university

결론

- 딥러닝 모델을 사용한 영상처리의 최근 발전 소개하였다.
- 딥러닝 기본 개념, 전통적인 ML과 다른 점, 딥러닝이 필요한 이유에 대해 설명했다.
- 복잡한 영상처리 및 컴퓨터 비전 작업을 해결하기 위해 특별히 고안된 심층신경망으로서 CNN 소개하였다.
- 컨볼루션, 풀링, FC 계층이 있는 CNN 아키텍쳐를 논의하였다.
- 파이썬에 널리 사용되는 두가지 라이브러리 Tensorflow와 Keras를 알아보았다.
- 필기숫자분류에서 MNIST 데이터 셋의 테스트 정확도를 증가시키기 위해
  - FC계층만을 사용하는 방법
  - CNN을 사용하는 방법

을 보여주었다.

- VGG-16/19, GoogleNet, ResNet과 같은 인기있는 네트워크에 대해 살펴보았다.
- Keras의 VGG-16모델은 Kaggle의 개 대 고양이 대회 영상으로 학습하고, 검증 데이터 셋에서 적절한 정확도를 실행되는가 확인하였다.

동국대학교
dongguk university